

Java Stream API

Practice Questions & Solutions

Advanced Mapping / FlatMap Operations / Sorting Complex

1. Advanced Mapping

Q1. Extract names of employees earning more than average salary.

Solution:

```
import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
}

public class AboveAverageSalary {
    public static void main(String[] args) {
        List<Employee> employees = List.of(
            new Employee("Alice", 55000),
            new Employee("Bob", 42000),
            new Employee("Charlie", 68000),
            new Employee("David", 30000),
            new Employee("Eva", 75000)
        );

        double avgSalary = employees.stream()
            .mapToDouble(e -> e.salary)
            .average()
            .orElse(0.0);

        List<String> aboveAverage = employees.stream()
            .filter(e -> e.salary > avgSalary)
            .map(e -> e.name)
            .collect(Collectors.toList());

        System.out.println("Average Salary: " + avgSalary);
        System.out.println("Employees above average: " + aboveAverage);
    }
}
```

Output: Average Salary: 54000.0, Employees above average: [Alice, Charlie, Eva]

Q2. Get the length of the longest string in a list.

Solution:

```
import java.util.*;
import java.util.stream.*;

public class LongestStringLength {
    public static void main(String[] args) {
```

```

        List<String> words = List.of("apple", "banana", "kiwi", "watermelon", "fig");

        int maxLength = words.stream()
            .mapToInt(String::length)
            .max()
            .orElse(0);

        System.out.println("Length of longest string: " + maxLength);
    }
}

```

Output: Length of longest string: 10 ("watermelon")

Q3. Convert a list of strings to a map (string → length).

Solution:

```

import java.util.*;
import java.util.stream.*;

public class StringToLengthMap {
    public static void main(String[] args) {
        List<String> words = List.of("apple", "banana", "kiwi", "fig");

        Map<String, Integer> lengthMap = words.stream()
            .collect(Collectors.toMap(
                word -> word,
                String::length
            ));

        System.out.println(lengthMap);
    }
}

```

Output: {banana=6, apple=5, kiwi=4, fig=3} (order may vary with HashMap)

Q4. Convert a list of products to a map (product name → price).

Solution:

```

import java.util.*;
import java.util.stream.*;

class Product {
    String name;
    double price;

    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}

public class ProductToPriceMap {
    public static void main(String[] args) {

```

```

        List<Product> products = List.of(
            new Product("Laptop", 55000.0),
            new Product("Mouse", 500.0),
            new Product("Keyboard", 1200.0)
        );

        Map<String, Double> priceMap = products.stream()
            .collect(Collectors.toMap(p -> p.name, p -> p.price));

        System.out.println(priceMap);
    }
}

```

Output: {Laptop=55000.0, Mouse=500.0, Keyboard=1200.0}

Q5. Transform a list of employees to a list of their departments (unique).

Solution:

```

import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    String department;

    Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }
}

public class UniqueDepartments {
    public static void main(String[] args) {
        List<Employee> employees = List.of(
            new Employee("Alice", "HR"),
            new Employee("Bob", "IT"),
            new Employee("Charlie", "IT"),
            new Employee("David", "Finance"),
            new Employee("Eva", "HR")
        );

        List<String> departments = employees.stream()
            .map(e -> e.department)
            .distinct()
            .collect(Collectors.toList());

        System.out.println("Unique Departments: " + departments);
    }
}

```

Output: Unique Departments: [HR, IT, Finance]

2. FlatMap Operations

Q6. Flatten a list of lists of integers.

Solution:

```
import java.util.*;
import java.util.stream.*;

public class FlattenListOfLists {
    public static void main(String[] args) {
        List<List<Integer>> numberLists = List.of(
            List.of(1, 2, 3),
            List.of(4, 5),
            List.of(6, 7, 8, 9)
        );

        List<Integer> flatList = numberLists.stream()
            .flatMap(List::stream)
            .collect(Collectors.toList());

        System.out.println(flatList);
    }
}
```

Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Q7. Flatten a list of arrays of strings.

Solution:

```
import java.util.*;
import java.util.stream.*;

public class FlattenArraysOfStrings {
    public static void main(String[] args) {
        List<String[]> arrayList = List.of(
            new String[]{"a", "b"},
            new String[]{"c", "d", "e"},
            new String[]{"f"}
        );

        List<String> flatList = arrayList.stream()
            .flatMap(Arrays::stream)
            .collect(Collectors.toList());

        System.out.println(flatList);
    }
}
```

Output: [a, b, c, d, e, f]

Q8. Split sentences into individual words and collect all unique words.

Solution:

```
import java.util.*;
import java.util.stream.*;
```

```

public class UniqueWords {
    public static void main(String[] args) {
        List<String> sentences = List.of(
            "Java is powerful",
            "Java is easy to learn",
            "Streams make Java powerful"
        );

        Set<String> uniqueWords = sentences.stream()
            .flatMap(sentence -> Arrays.stream(sentence.split("\\s+")))
            .collect(Collectors.toCollection(TreeSet::new));

        System.out.println(uniqueWords);
    }
}

```

Output: [Java, Streams, easy, is, learn, make, powerful, to]

Q9. Given a list of orders (each with list of items), get all item names.

Solution:

```

import java.util.*;
import java.util.stream.*;

class Order {
    int orderId;
    List<String> items;

    Order(int orderId, List<String> items) {
        this.orderId = orderId;
        this.items = items;
    }
}

public class AllItemNames {
    public static void main(String[] args) {
        List<Order> orders = List.of(
            new Order(1, List.of("Pen", "Notebook")),
            new Order(2, List.of("Laptop", "Mouse", "Pen")),
            new Order(3, List.of("Bag"))
        );

        List<String> allItems = orders.stream()
            .flatMap(order -> order.items.stream())
            .collect(Collectors.toList());

        System.out.println("All items: " + allItems);

        Set<String> uniqueItems = orders.stream()
            .flatMap(order -> order.items.stream())
            .collect(Collectors.toSet());

        System.out.println("Unique items: " + uniqueItems);
    }
}

```

```
}  
}
```

Output: All items: [Pen, Notebook, Laptop, Mouse, Pen, Bag]

Q10. Flatten a map of lists into a single list.

Solution:

```
import java.util.*;  
import java.util.stream.*;  
  
public class FlattenMapOfLists {  
    public static void main(String[] args) {  
        Map<String, List<Integer>> map = new LinkedHashMap<>();  
        map.put("A", List.of(1, 2, 3));  
        map.put("B", List.of(4, 5));  
        map.put("C", List.of(6, 7, 8));  
  
        List<Integer> flatList = map.values().stream()  
            .flatMap(List::stream)  
            .collect(Collectors.toList());  
  
        System.out.println(flatList);  
    }  
}
```

Output: [1, 2, 3, 4, 5, 6, 7, 8]

3. Sorting Complex

Q11. Sort strings by length, then alphabetically.

Solution:

```
import java.util.*;  
import java.util.stream.*;  
import java.util.Comparator;  
  
public class SortByLengthThenAlpha {  
    public static void main(String[] args) {  
        List<String> words = new ArrayList<>(List.of(  
            "banana", "kiwi", "fig", "apple", "pear", "date"  
        ));  
  
        List<String> sorted = words.stream()  
            .sorted(Comparator.comparingInt(String::length)  
                .thenComparing(Comparator.naturalOrder()))  
            .collect(Collectors.toList());  
  
        System.out.println(sorted);  
    }  
}
```

Output: [fig, date, kiwi, pear, apple, banana]

Q12. Sort numbers by their absolute value.

Solution:

```
import java.util.*;
import java.util.stream.*;
import java.util.Comparator;

public class SortByAbsoluteValue {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>(List.of(
            -8, 3, -1, 5, -10, 2
        ));

        List<Integer> sorted = numbers.stream()
            .sorted(Comparator.comparingInt(Math::abs))
            .collect(Collectors.toList());

        System.out.println(sorted);
    }
}
```

Output: [-1, 2, 3, 5, -8, -10]